

- `maxOrNull()` - Maximum in collections
- `minOrNull()` - Minimum in collections.

* Jetpack Compose:

* Gradle:

- plays a major role. It is just a task runner. Gradle is also responsible for signing and launching the app. But Gradle doesn't do it and it knows the tools that do it.
- `/gradlew task` = shows all the tasks gradle does.
- `build.gradle.kts` (for project) = top level file where we can add configurations.
- `build.gradle.kts (:app)` = build automatically by Android Studio.
- Plugins = used in `build.gradle.kts`. Nothing but a Kotlin class. Each plugin has an id and when we don't want to apply it we write:

plugins {

alias(plugin) apply false

}

- `build.gradle.kts (:app)` has:

- `plugin` = above

- `android` = only applies to android

- `namespace` = package name

- `compiler` = android version code should compile to.

- default config = default configuration of the app.
- minSDK = minimum api the app runs on
- target SDK = highest SDK level our app is intended for.
- versionCode = the version. used to detect an update.
- versionName = human readable version name.

test InstrumentationRunner =

- Compile Option = how the app should be compiled.
- ~~Kotlin~~ Kotlin option = same as compile option.
- dependencies = library you want to include.

- Every module/library comes with a gradle file. It is so because the module may require different dependencies than our app gradle.
- library notation:

dependencies {

implementation (project(":libraryname"))

}

without this, we would not ~~not~~ be able to use the library or anything related into our main activity.

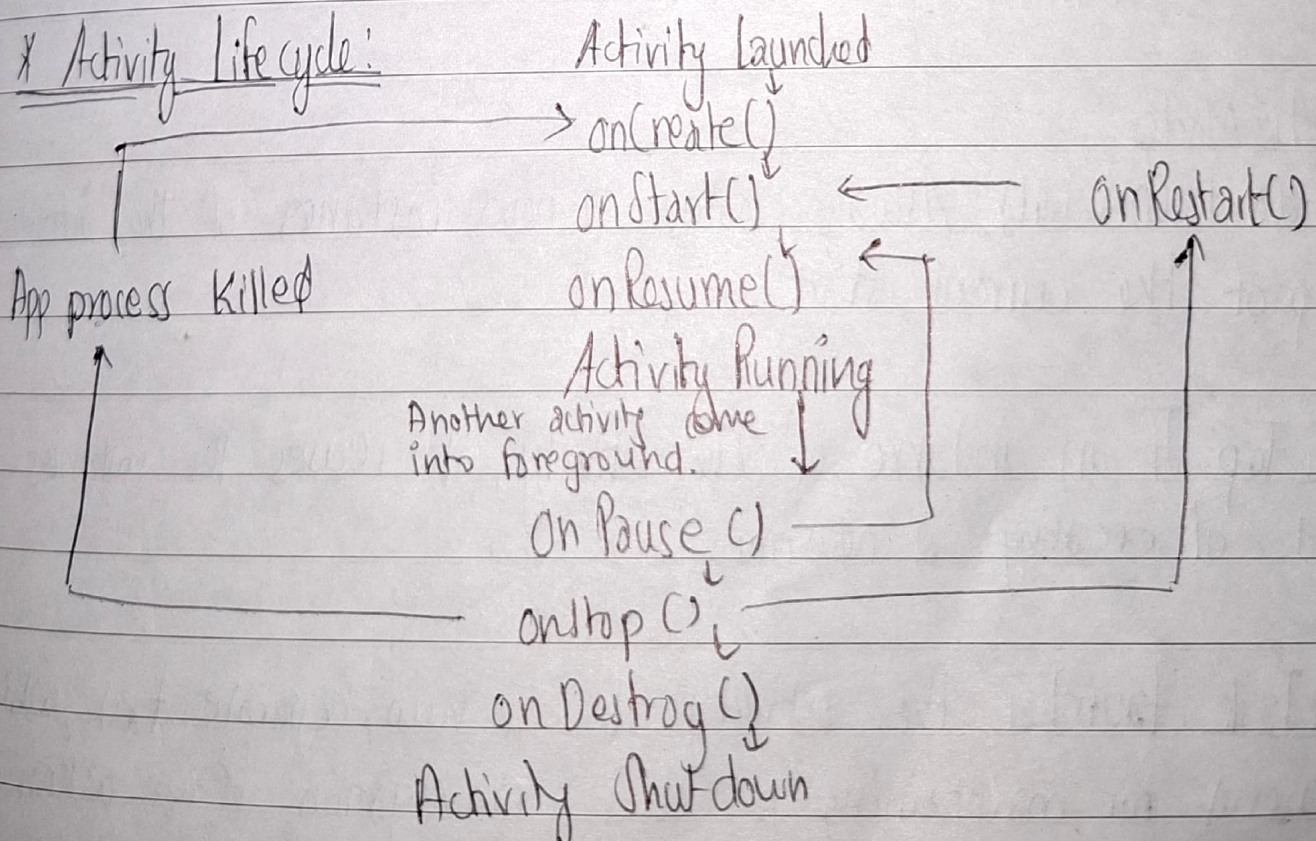
- There will be logic and configuration that are different in all of the modules and some that are same.
 - ~~Be~~ build type = stage where app is in during development.
- two build type: Debug and Release.
- isMinifyEnabled = makes code harder to understand when trying to reverse engineer.

- We can also create a new buildType using `create()`.
- We can also define
- `BuildConfigField()` = builds primitive field you can access in your build, that still might differ with build type.
- We need to enable `buildConfig()` in `buildFeatures()`
- `ProductFlavors()` = `BuildConfigField()` simple to build type but for release where there may be many type of user.
- `SourceSet` are the files gradle look for code.

* Activities:

- When we create a project in android studio, it will create a `MainActivity.kt` automatically.
- Activity is a component for one or more screen in our apps. and the unit users interact with.

* Activity Life cycle:



- `onStop()` and `onPause()` have similar use but `onPause` is used when the activity is still in user view.
- `onCreate()` - activity is created
- `onStart()` - activity is opened / started.
- `onResume()` - for when activity is paused.
- `onPause()` - when in view but not active
- `onStop()` - when running is background
- `onDestroy()` when closed.

* Tasks & Backstack:

- Stack ~~created~~ of activity created when moving from one activity to another so we can return back to the activity is known as Backstack.
- Task is the collection of multiple screen that belong together along with the stack.
- Android will remember all the tasks.

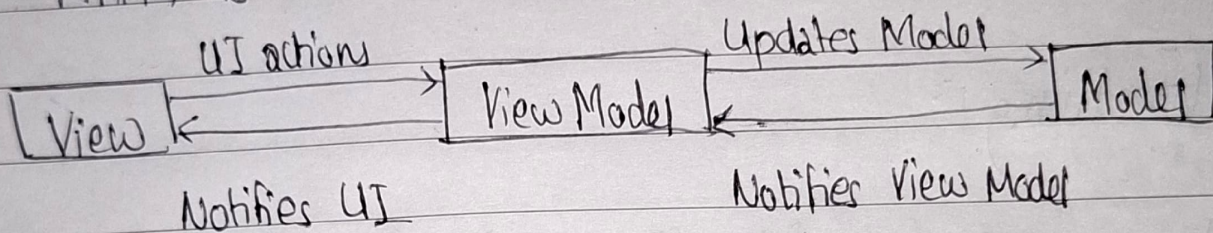
* Launch Modes:

- Standard (Default): Always creates a new instances of the activity on top of the current stack.
- Single Top: If an instance of the activity, it reuses the instances instead of creating a new one.
- Single Task: launches the activity in a new, separate task, which is useful for maintaining a clean navigation flow when jumping between apps.

• Single Instance: Similar to single task, but the activity is completely isolated, meaning no other ~~task~~ activities can be opened ~~to~~ within the same task. This is typically reserved for specialized use cases like payment processing.

* ViewModels:

• MVVM is the view model used in android. (Model View VM)



• VM acts as a bridge between View (UI) and Model (data source). Its primary job is to hold and prepare data for ~~the~~ the UI and handle user actions.

• VM is essential in Android because activities are destroyed and recreated during configuration change and VM classes survive these lifecycle event.

* Basic AD

1) Create a Project using Android Developers

2) Edit Main Activity.kt only for now.

3) maxres class and then fun in it

(rough)

a	b	c	d
e	f	g	h

P.T.O

~~Get~~ sort
 decreasing order sort
 compare

Pizza

Bi

file handling :

~~12 write~~ val f = file("name")
~~can file~~, writeText("Input") (create or replace)
 file.appendText() (add new info) ^{can}
 val content = file.readText() (as a single str)
 val ~~can~~ = file.readLine() (as a list by line)

o interface.

* Interactor:

